Internet of Things: Protocols and Networks

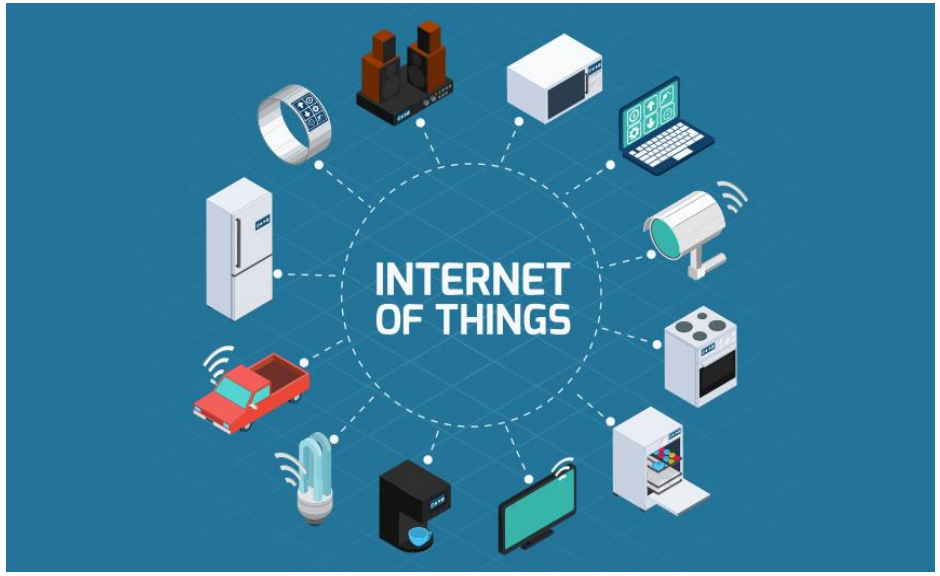
(CSC2106)

Edge Computing

How Edge, Fog, and Cloud Work Together in AIoT Systems

AIoT: Fusing AI with IoT – Relies on Edge Computing

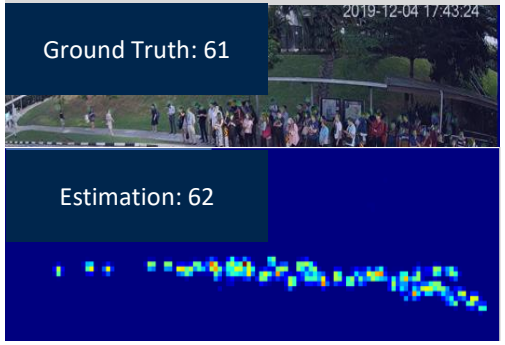
Traditional IoTs



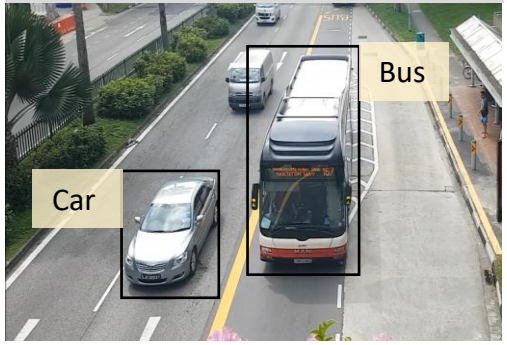
Source: https://www.bankinfosecurity.com/gao-assesses-iot-cybersecurity-other-risks-a-9926

AIoTs

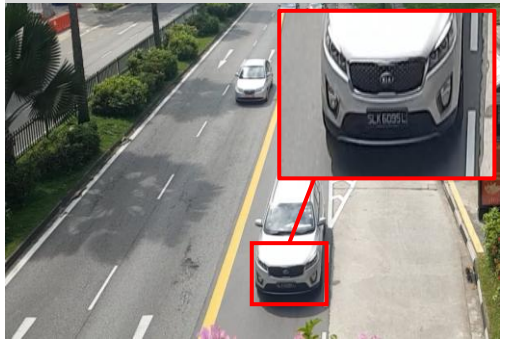
Crowd Counting



Object Localization & Classification

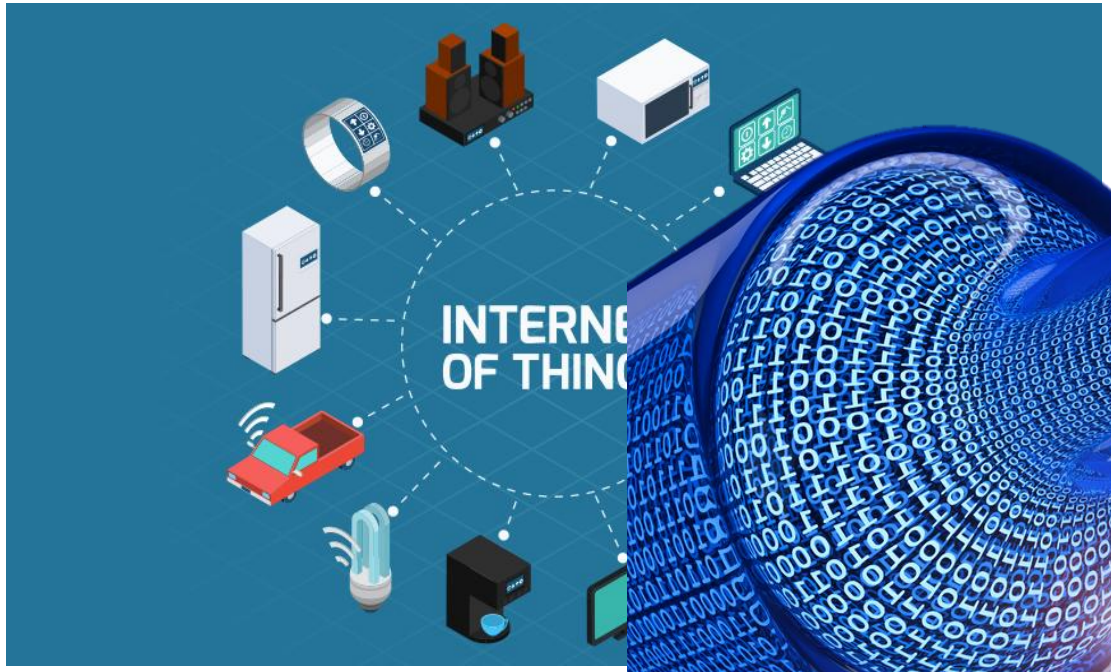


License Pate Recognition

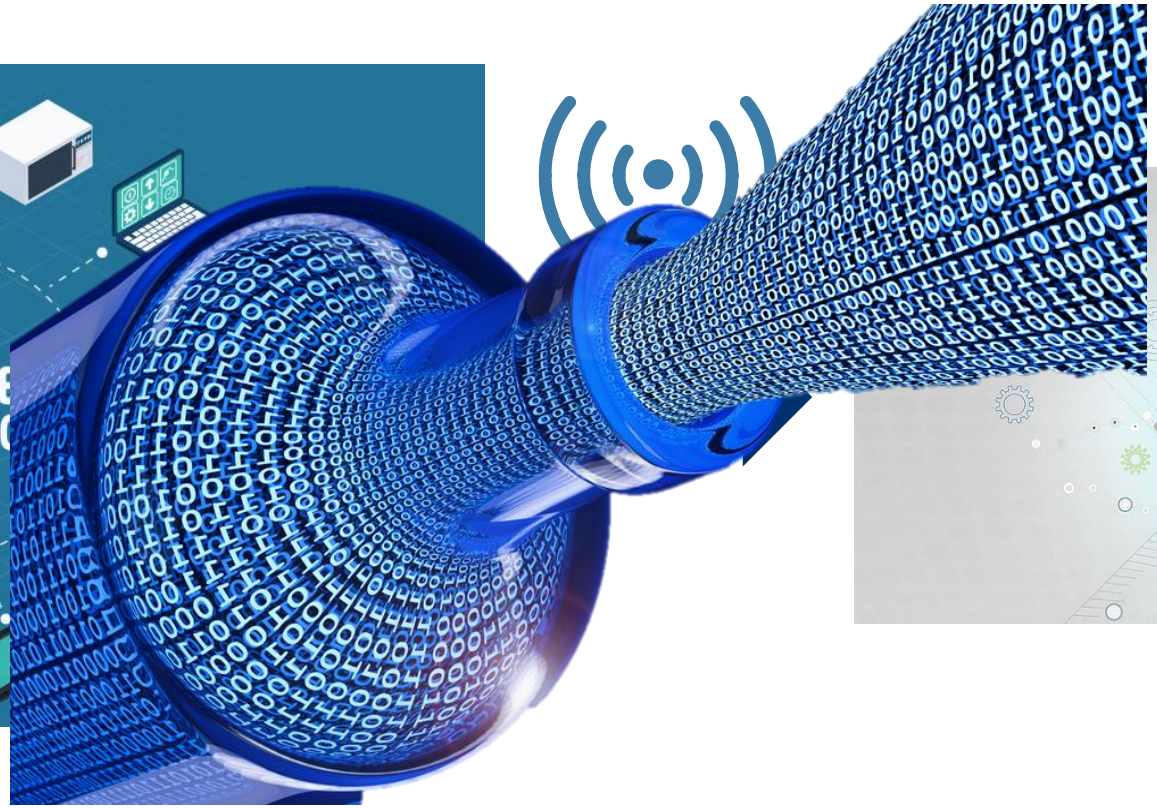


The Need for Edge Computing

- IDC estimates that there will be **41.6 billion connected IoT** devices generating a whopping **79.4 zettabytes of data** in 2025[1].



Source: https://www.bankinfosecurity.com/gao-assesses-iot-cybersecurity-other-risks-a-9926

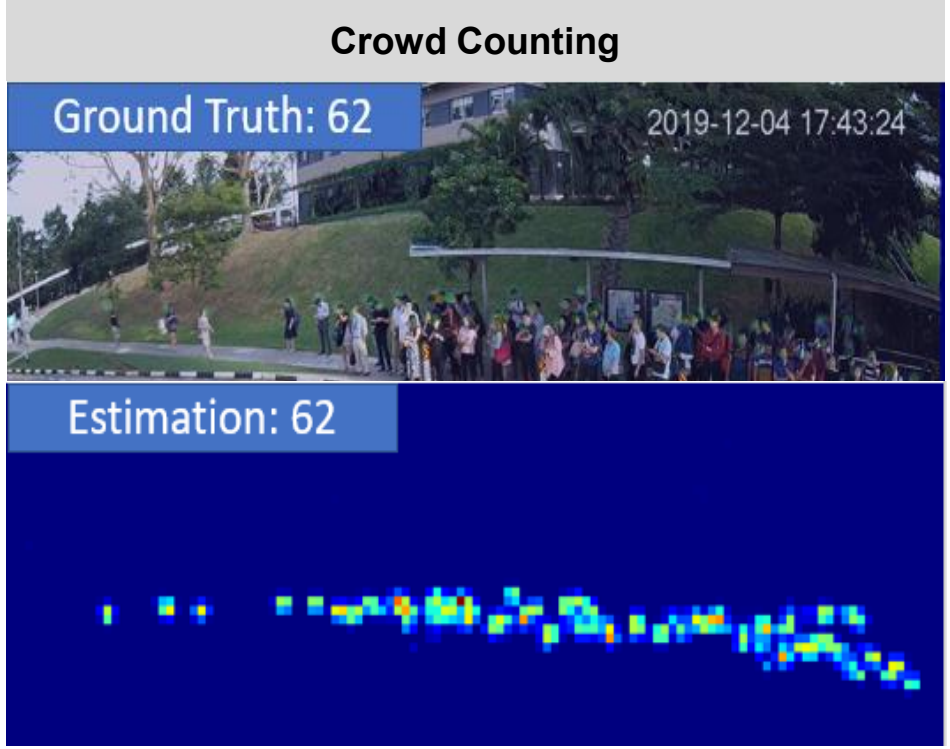


Source: https://www.hpcwire.com/

[1] Worldwide Global DataSphere IoT device and data forecast (2019-2023), IDC

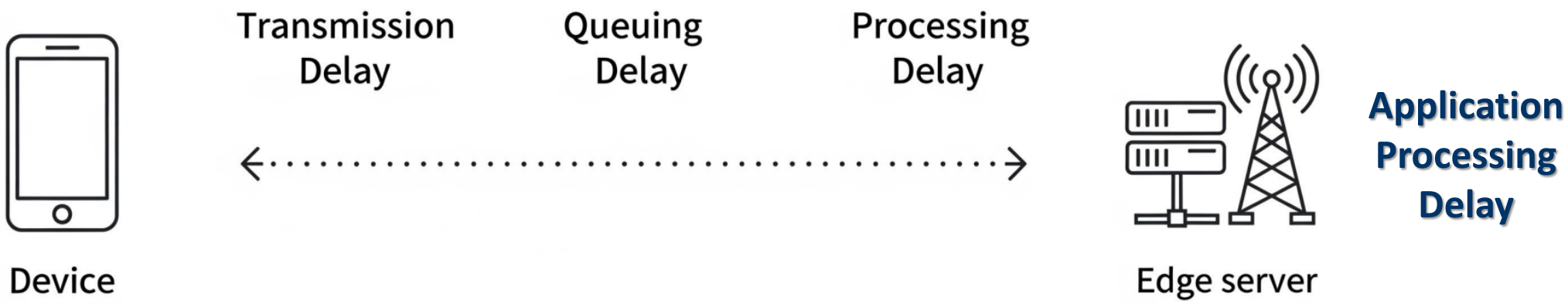
Why Edge Computing Matters in IoT Communications

- Communication **Bottlenecks**
  - Latency-sensitive tasks (e.g., camera detection).
  - Bandwidth limits, cost of raw telemetry.
  - Reliability issues in mobile/industrial deployments.
- **Benefits** of Edge Computing
  - Reduces **bandwidth** consumption
  - Reduces **latency**
  - Reduces **storage** requirements



Various Types of Latency

Communications latency is about reliability (are packets dropped?) and jitter (is the delay consistent?).

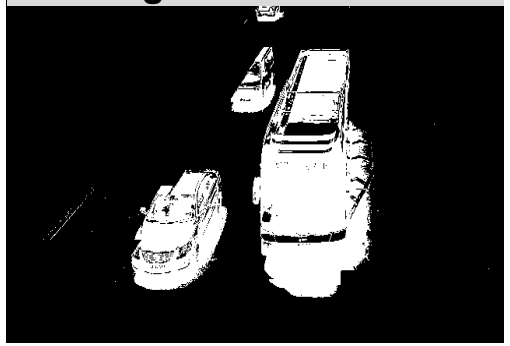


- **Key Components of Network Latency**:
  - **Transmission Delay**: Time to push all the packet's bits onto the link.
  - **Queuing Delay**: Time spent waiting in router queues.
  - **Processing Delay**: Time for a router to process the packet header.

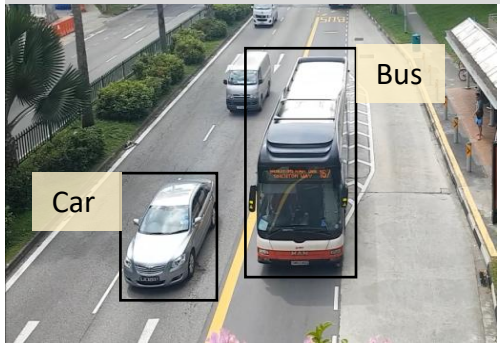
- **Key Components of Application Processing Latency**:
  - Running **inference** on an edge GPU
  - **Executing** business logic
  - Handling **service** requests
  - Doing sensor fusion, **analytics**, or aggregation

Example Application: Illegal Parking Detection

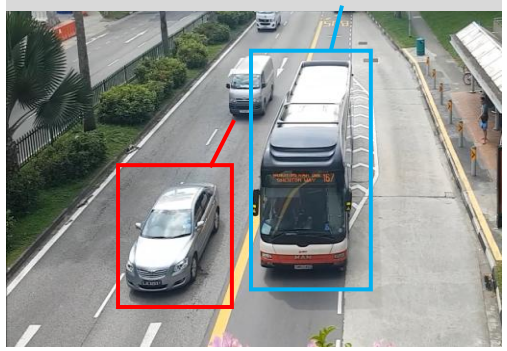
Pre-processing Background Modeling & Foreground Detection



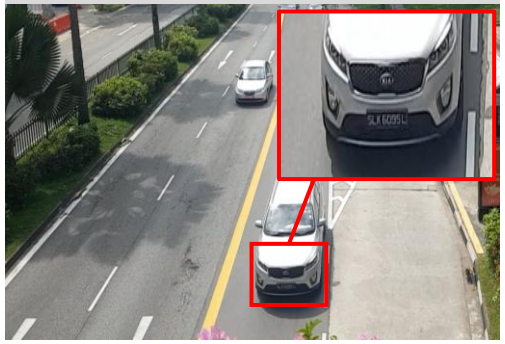
Classification



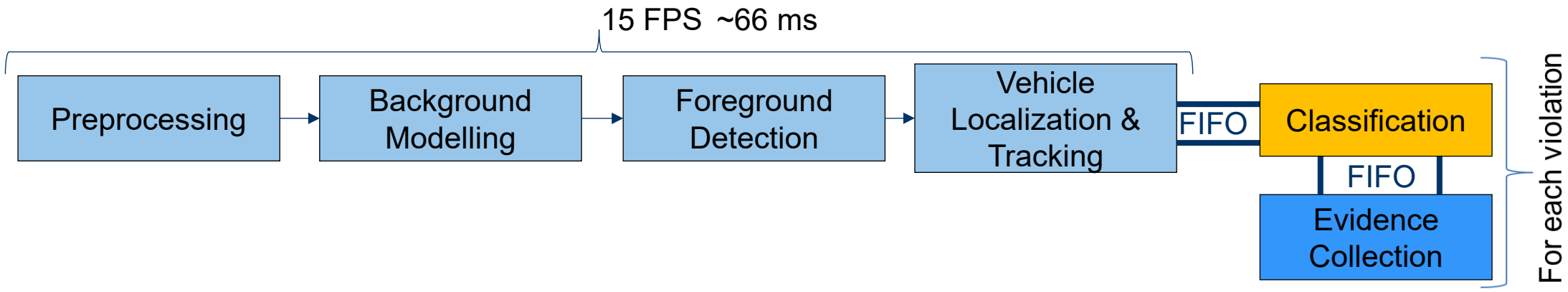
Vehicle Localization & Tracking



Evidence Collection

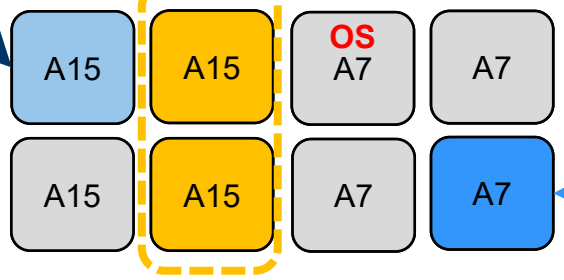


Performance Analysis and Mapping – Illegal Parking



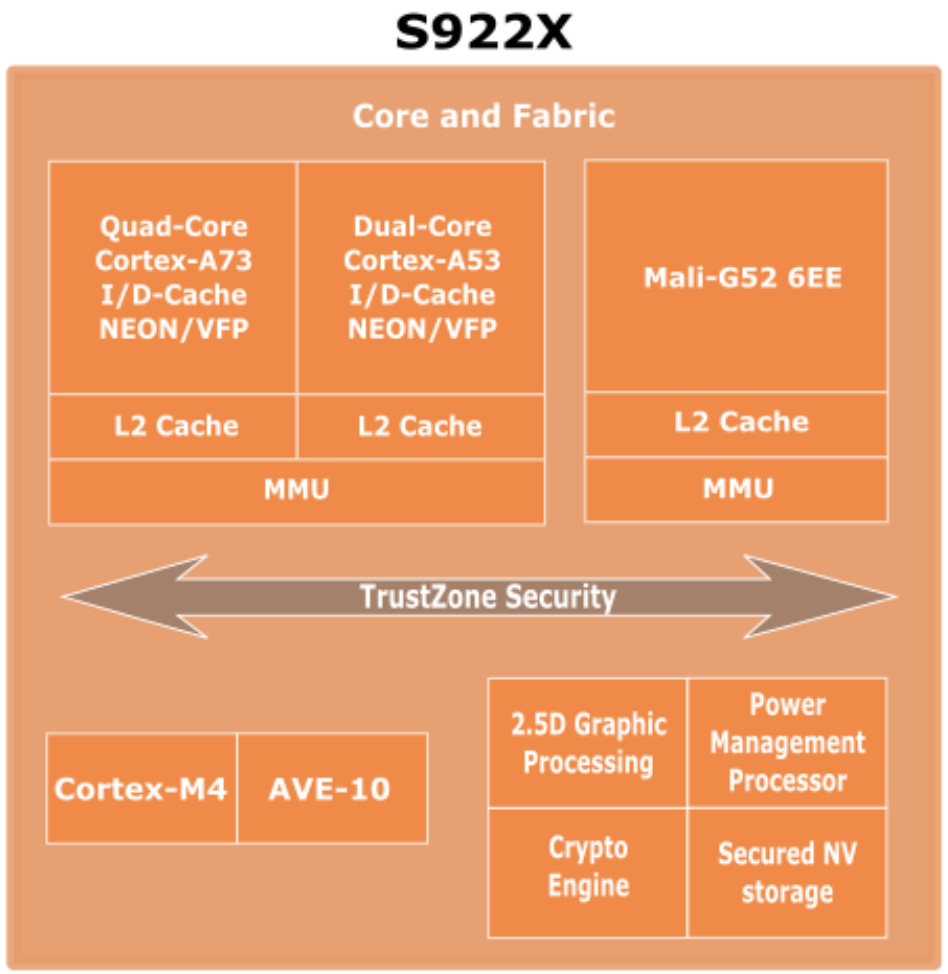
Preprocessing	Background Modelling & Foreground detection	Vehicle Localization & Tracking	Classification	Evidence Collection
3.3 ms on one A15 30 ms on one A7	6 ms on one A15 50 ms on one A7	12 ms on one A15 80 ms on one A7	500 ms on one A15 350 ms on two A15	20 ms on one A15 100 ms on one A7

Given a power budget, a possible mapping could be:



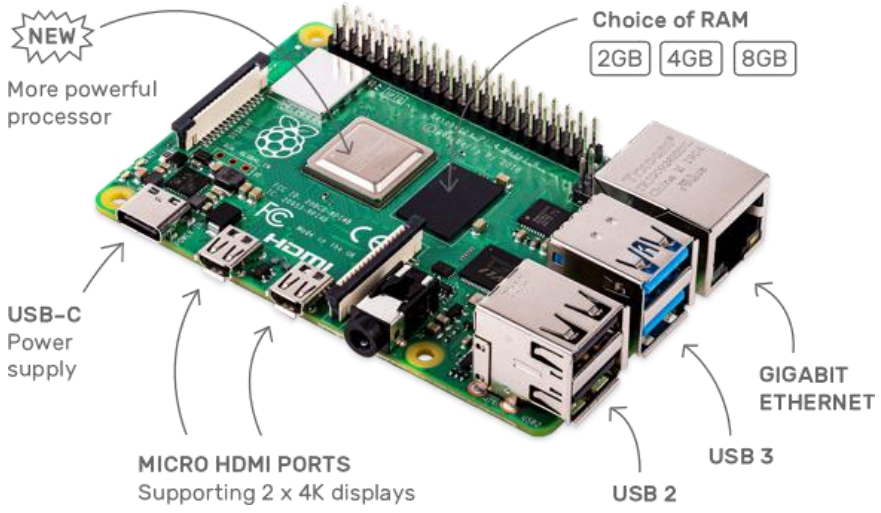
ODROID-XU3

Edge Devices – SBC Centric



6-core CPU from Odroid N2+

Raspberry Pi 4



Specifications

- Broadcom BCM2711, Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.5GHz
- 2GB, 4GB or 8GB LPDDR4-3200 SDRAM (depending on model)
- 2.4 GHz and 5.0 GHz IEEE 802.11ac wireless, Bluetooth 5.0, BLE
- Gigabit Ethernet
- 2 USB 3.0 ports; 2 USB 2.0 ports.
- Raspberry Pi standard 40 pin GPIO header (fully backwards compatible with previous boards)

Edge Devices – GPU Centric

Compare NVIDIA Jetson Module Specifications

	Jetson Nano	Jetson TX2 Series				Jetson Xavier NX	Jetson AGX Xavier
		TX2 NX	TX2 4GB	TX2	TX2i		
AI Performance	472 GFLOPS	1.33 TFLOPS			1.26 TFLOPS	21 TOPS	32 TOPS
GPU	128-core NVIDIA Maxwell™ GPU	256-core NVIDIA Pascal™ GPU				384-core NVIDIA Volta™ GPU with 48 Tensor Cores	512-core NVIDIA Volta™ GPU with 64 Tensor Cores
CPU	Quad-core ARM® Cortex®-A57 MPCore processor	Dual-core Denver 2 64-bit CPU and quad-core Arm® Cortex®-A57 MPCore processor				6-core NVIDIA Carmel ARM®v8.2 64-bit CPU 6MB L2 + 4MB L3	8-core NVIDIA Carmel Arm®v8.2 64-bit CPU 8MB L2 + 4MB L3
Memory	4 GB 64-bit LPDDR4 25.6GB/s	4 GB 128-bit LPDDR4 51.2GB/s		8 GB 128-bit LPDDR4 59.7GB/s	8 GB 128-bit LPDDR4 (ECC Support) 51.2GB/s	8 GB 128-bit LPDDR4x 51.2GB/s	32 GB 256-bit LPDDR4x 136.5GB/s
Storage	16 GB eMMC 5.1	16 GB eMMC 5.1		32 GB eMMC 5.1	32 GB eMMC 5.1	16 GB eMMC 5.1	32GB eMMC 5.1
Power	5W | 10W	7.5W | 15W			10W | 20W	10W | 15W	10W | 15W | 30W

Edge Devices – GPU Centric (updated)

Compare NVIDIA Jetson **Orin** Module Specifications

Feature	**Jetson Orin Nano 4GB**	**Jetson Orin Nano 8GB**	**Jetson Orin NX 8GB**	**Jetson Orin NX 16GB**	**Jetson AGX Orin 32GB**	**Jetson AGX Orin 64GB**
AI Performance	Up to **20 TOPS** (INT8 Sparse)	Up to **40 TOPS** (INT8 Sparse)	**70 TOPS** (INT8)	**100 TOPS** (INT8)	Up to **200+ TOPS**	Up to **275 TOPS**
GPU	512-core NVIDIA Ampere GPU + 16 Tensor Cores	1024-core NVIDIA Ampere GPU + 32 Tensor Cores	1024-core NVIDIA Ampere GPU + 32 Tensor Cores	1024-core NVIDIA Ampere GPU + 32 Tensor Cores	1792-core Ampere GPU + 56 Tensor Cores	2048-core Ampere GPU + 64 Tensor Cores
CPU	6-core Arm Cortex-A78AE	6-core Arm Cortex-A78AE	6-core Arm Cortex-A78AE	8-core Arm Cortex-A78AE	8-core Arm Cortex-A78AE	12-core Arm Cortex-A78AE
Memory	4GB 64-bit LPDDR5	8GB 128-bit LPDDR5	8GB 128-bit LPDDR5 (102.4 GB/s)	16GB 128-bit LPDDR5 (102.4 GB/s)	32GB 256-bit LPDDR5	64GB 256-bit LPDDR5
Storage	External NVMe	External NVMe	External NVMe	External NVMe	64GB eMMC 5.1 + NVMe	64GB eMMC 5.1 + NVMe
Power	5W / 10W	7W / 15W	10W / 20W	10W / 25W	15–40W	15–60W

Edge Devices – MiniPC Centric

Compare **AMD-based** Specifications

APU Family	NPU TOPS	Total AI TOPS (CPU+GPU+NPU)	GPU	Notes
Ryzen AI Max+ 395	~50	~120–126	Radeon 8060S	Highest overall performance; huge GPU compute
Ryzen AI 300 (HX 370 / 350 / 340)	~50	~70–80	Radeon 800M	Compact edge AI units; second-gen XDNA2 NPUs
Ryzen 8040 (8840U / 8640U)	~16	~30–39	Radeon 780M	Strong mid-range AI PCs with dual USB4 support
Ryzen 8845HS / 7840HS / 7940HS	~16	~35–40	Radeon 780M	Older but still potent; excellent GPU inference
Phoenix 7040 (early Ryzen AI)	~10–16	<30	Radeon 780M	Entry-level NPU + solid GPU; widely available

EDGE AI ECOSYSTEM

DNN network architecture

- Lightweight CNNs (EfficientNet-Lite, MobileNetV3/V4, EdgeNeXt
- Efficient CNN Scaling Families (EfficientNet (B0–B7)
- Real-Time Object Detectors (YOLO family)
- Efficient Vision Transformers (MobileViT, EdgeViT, Mobile-optimized DeiT)
- Lightweight Segmentation Models (MobileSAM, FastSAM, SAM derivatives)

DNN framework (training, inference)

- PyTorch (mainstream for training)
- TensorFlow 2.xJAX (for experimental/ matrix-heavy workloads)
- ONNX (critical for compatibility)

Inference Engines

- **ONNX Runtime** - ROCm (AMD), CUDA (NVIDIA), DirectML (Windows GPU/NPU), Core ML (Apple ANE), OpenVINO (Intel CPU/NPU), TensorRT, etc.
- **TensorRT** (NVIDIA), OpenVINO (Intel), TFLite (CPU/Edge TPUs), MIGraphX (AMD ROCm)

- *(Weight Compression, Filter Pruning, Quantisation)*

Edge Deployment Tools

- DeepStreamEdge
- TPU Compiler
- TFLite Converter
- OpenVINO suite

Hardware

- **CPUs**: ARM Cortex-A/M, RISC-V (gaining traction)
- **GPUs**: Nvidia Jetson Orin, Mali-G710+, Imagination GPUs
- **NPUs/Accelerators**: Apple Neural Engine, Hailo, Google TPU 2.0
- **SoCs**: Snapdragon AI, Dimensity 9000 series, RK3588

Hardware Programming Languages

Multi-core CPU

- OpenMP, NEON (extension)

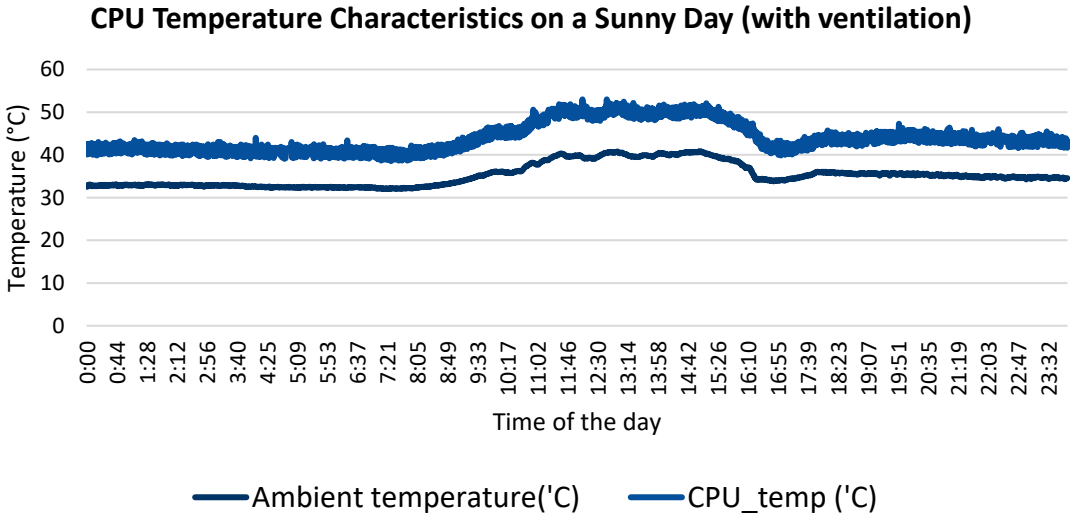
GPU

- **NVIDIA**: cuDNN, cuBLAS, cuFFT
- **AMD**: MIOpen, rocBLAS, rocFFT, rocRAND

Real life Deployment Challenges

- Waterproofing requirements
- Thermal Challenges
- Waterproofing and thermal are conflicting requirements
- Temperature inside the box can rise over 65°C during daytime, without any ventilation
- And CPU Temperature is even higher than that. Can easily rise to 85°C during daytime

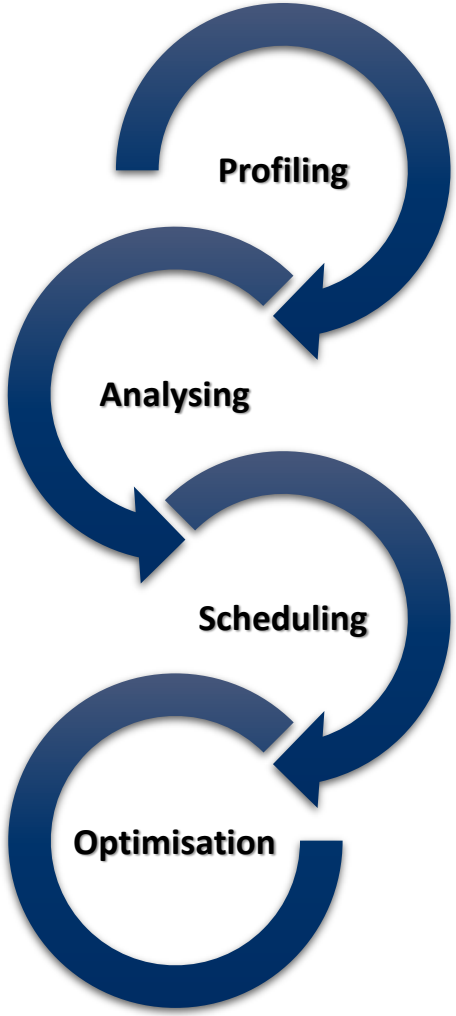
Traffic Junction



Motivation - The Need for Optimisation

Aspect	Model-Centric Optimisation	System-Level Optimisation
Focus Area	Neural network architecture and parameters	Hardware, OS, memory, and data movement
Objective	Reduce model size, complexity, and computation	Maximize hardware utilization and system efficiency
Techniques	Pruning, Quantization, Knowledge Distillation, NAS	I/O Optimization, Memory Management, Scheduling
Benefits	Lower memory and computation costs	Faster and more efficient inference
Challenges	Maintaining accuracy after optimization	Balancing resource contention and system overhead

Deployment Optimisation Process



Aspect	Techniques	Benefits
Advanced Profiling	Fine-grained performance analysis, Memory access patterns	Identifies bottlenecks and guides optimizations
Analysing Bottlenecks	CPU/GPU utilization, Memory/I/O, OS-level	Maximizes hardware utilization
Dynamic Scheduling	Task scheduling, Model partitioning, Adaptive precision	Reduces latency and improves resource efficiency
Advanced Optimization	Hardware acceleration, Code optimization, Graph optimization	Accelerates computation and minimizes power usage

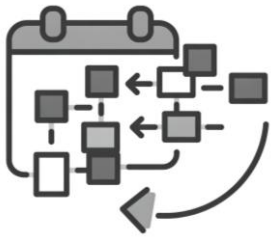
Advanced Profiling Techniques

Technique	Description	Tools/Examples	Benefits
Model Performance Analysis	Analyzes execution time of individual layers, operations, and functions. Identifies bottlenecks like convolutions or activation functions.	- **TensorBoard Profiler**: Layer-wise execution time and memory usage. - **NVIDIA Nsight Systems**: GPU-specific profiling for CUDA operations.	Pinpoints the most time-consuming layers for targeted optimizations.
Memory Analysis	Examines memory access patterns to detect cache misses and inefficient data movement.	- **Valgrind's Cachegrind**: Analyses cache performance. - **ARM Streamline**: Memory usage profiling on ARM devices.	Optimizes data locality and cache usage, reducing memory bottlenecks.
Power Profiling	Measures power consumption to identify energy-intensive operations. Essential for battery-powered edge devices.	- **ARM Energy Probe** - **Intel Power Gadget**	Optimizes power usage and scheduling strategies for energy efficiency.
System-Level Profiling	Analyzes low-level metrics such as cycles, instructions, cache misses and branch mispredictions.	- **perf**: Profiling tool for Linux systems, e.g. *perf stat -e cycles, instructions my_program*	Provides deep insights into CPU and hardware behavior (system-level bottlenecks) beyond model-specific issues.
Network Profiling	Captures and analyzes network traffic to detect latency, bandwidth limitations, and packet loss.	- **tcpdump**, **Wireshark** - Network latency and bandwidth usage diagrams.	Identifies network-related performance issues in edge-cloud communication.

Analysing System-Level Bottlenecks

Aspect	Description	Tools/Examples	Optimization Approach
CPU/GPU Utilization	Identifies under-utilization or overloading of processing units.	- **Perf** (Linux Performance Profiler) - **NVIDIA Visual Profiler (nvvp)**	• Load balancing between CPU and GPU. • Utilize heterogeneous computing (e.g., NPUs).
Memory and I/O	Analyzes latency due to memory access or data transfer operations.	- **gprof**, **Valgrind** - I/O solutions: Faster storage (e.g., SSD), buffering data	• Memory Pooling: Reuse memory buffers. • Layer Fusion: Reduce intermediate data movement. • Optimize data access patterns.
OS-Level	Examines the impact of OS scheduling, resource allocation, and context switching.	- **Zephyr RTOS** for real-time constraints - Lightweight runtime (e.g., TensorFlow Lite Micro)	• Real-Time Scheduling: Prioritize ML tasks. • Minimize OS overhead with lightweight runtimes.

Dynamic and Adaptive Scheduling



Technique	Description	Approach	Benefits	Examples
Dynamic Task Scheduling	Dynamically assigns tasks to processing units (CPU, GPU, NPU) based on resource availability.	- **Work Stealing Algorithms:** Idle processors take over tasks from busy ones. - **Priority-Based Scheduling:** Assigns priority levels to tasks (e.g., high, medium, low).	- Maximizes hardware utilization. - Reduces latency by prioritizing critical tasks.	- **TVM Runtime**: Dynamic scheduling for heterogeneous devices. - **Preemptive Scheduling**: Interrupts lower-priority tasks for high-priority ones.
Adaptive Model Partitioning	Splits model execution across multiple devices (e.g., edge and cloud) and adapts partitioning based on runtime conditions like network latency.	- **Edge-Cloud Offloading:** Executes latency-critical layers on the edge and offloads other layers to the cloud.	- Balances latency, energy, and computational load. - Improves performance under varying network conditions.	- **Neurosurgeon Framework:** Real-time model partitioning between edge and cloud.
Adaptive Resource Allocation	Dynamically allocates system resources (CPU, memory) based on current system load and task requirements.	- Allocates more resources to high-priority or critical tasks during peak load. - Minimizes resource allocation for background tasks during idle periods.	- Enhances system responsiveness and efficiency. - Ensures optimal resource usage without overloading.	- **Real-Time Operating Systems (RTOS):** Adaptive scheduling in time-critical applications.

Advanced Optimisation Techniques

Focus Area	Techniques	Examples
Hardware Acceleration	• **Operator Fusion:** Combines multiple operations into one kernel to minimize memory access. • **Domain-Specific Accelerators:** Custom accelerators for specific tasks (e.g., convolution, matrix multiplication).	• **Google Edge TPU:** Low-latency inference on edge devices. • **NVIDIA TensorRT:** GPU acceleration for deep learning models.
Code Optimization	• **Kernel Optimization:** Improves cache locality and reduces memory access. • **Loop Unrolling and Tiling:** Enhances parallelism and data locality. • **Vectorization:** Utilizes SIMD (Single Instruction, Multiple Data) for faster computation.	• **LLVM and GCC Compilers:** Low-level code optimization. • **XLA (Accelerated Linear Algebra) Compiler:** Efficient kernel generation in TensorFlow.
Graph-Level Optimization	• **Common Subexpression Elimination:** Avoids redundant computations. • **Constant Folding:** Pre-computes constant expressions during compilation.	• **TensorFlow XLA and TVM:** Perform graph-level optimisations for efficient execution.

Case Study: Smart Camera for Object Detection

- **Application:** An edge-based smart camera performing real-time object detection & alerting.
- **Bottleneck:** Camera transmits results with high latency to the cloud for processing.
- **Tools:** *tcpdump* and *Wireshark*, network traffic analysis revealed high transmission delays due to limited bandwidth and intermittent connectivity.
- **Possible Solution:**
  - *Edge-Cloud Partitioning:* Critical detection logic was moved to the edge device, sending only summarised alerts to the cloud, reducing data.
  - *Protocol Optimisation:* Switching from HTTP to MQTT for more efficient, lightweight communication.

Case Study: Real-time Face Recognition Smart Camera

Imagine: You have a home security camera that can recognise faces. It can tell if a family member is at the door or a stranger is approaching. The camera can quickly alert you if an unknown person is at your door, even if the internet is down.

Challenge: High latency and power consumption when running complex face recognition models entirely on the edge device.

Adaptive Model Partitioning:

- The camera's face recognition "brain" is split into two parts.
- **Local**: The camera uses its own "brain" to quickly analyse faces and see if they're familiar.
- **Cloud**: If the camera needs extra brainpower to recognise someone, it sends the video to a powerful computer in the cloud for help. This happens when the internet connection is strong, and the camera isn't too busy.

Why This is Cool:

- **Fast Response:** Critical layers run on the edge, ensuring quick response.
- **Saves Energy:** Heavy layers offloaded to the cloud, reducing power consumption on the edge device.
- **Reduces Cost:** Minimizes cloud usage by running lightweight layers locally.

Think of it like a security guard with a walkie-talkie. They can handle most situations themselves, but they can call for backup from the central office when needed!

IoT vs. IIoT

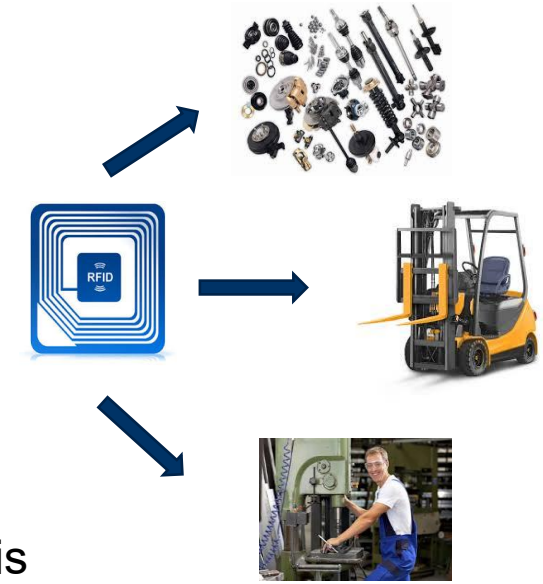
APPLICATION DOMAINS

ACCURACY AND PRECISION ARE TOP PRIORITIES FOR IIOT

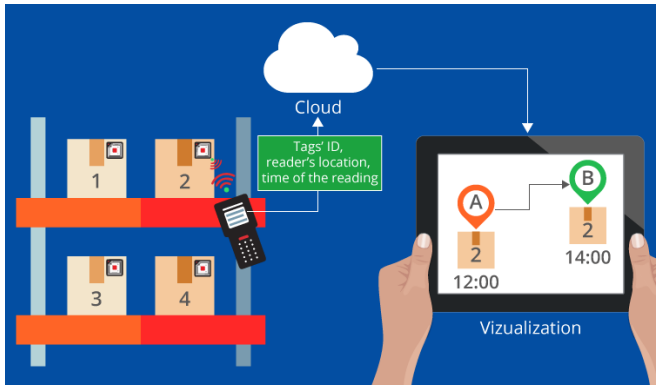
CYBER SECURITY IS CRITICAL

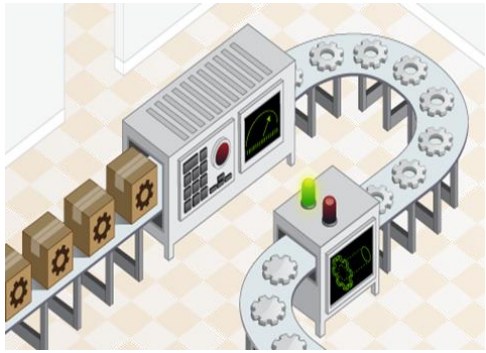
RELIABILITY AND ROBUSTNESS

Future Smart Factories



- Streamlined Factories
  - Asset tagging, locating, supply chain management
  - Easier to do Just-in-time asset management
- SMART Inventory management
  - Sensors on containers can determine when a product is running low
  - Automatic alerts to proactively re-order the parts or orders can be automatically placed with suppliers
- SMART Quality control
  - RFIDs attached to products can be used to tag defective products
  - Automatic alarms if failure rate crosses a certain threshold for early course correction



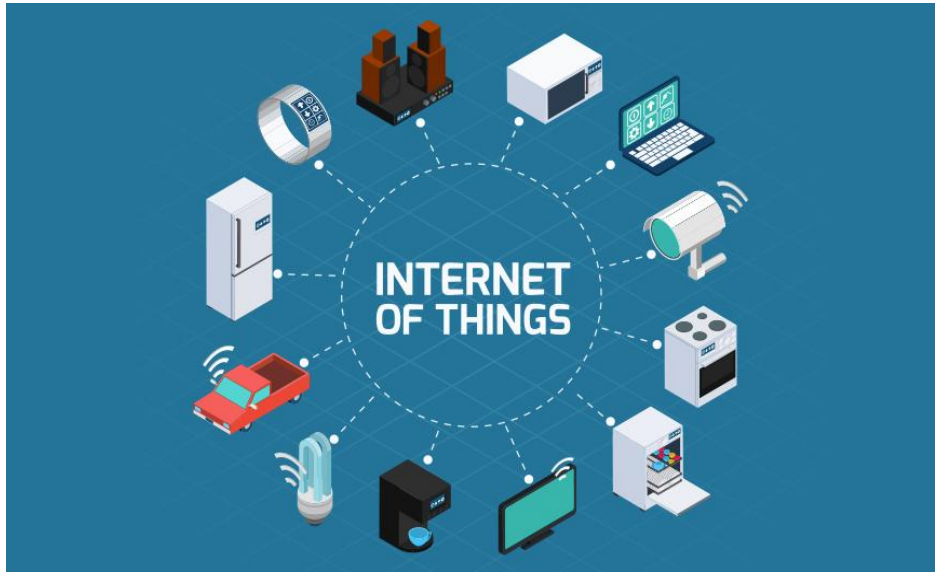


Source: Entrigna

Cloud Computing

Cloud vs Edge Computing

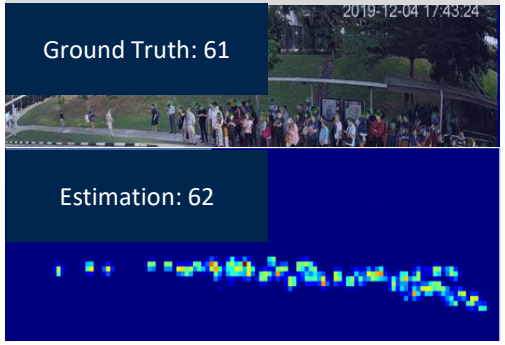
Traditional IoTs: Relies on Cloud Computing



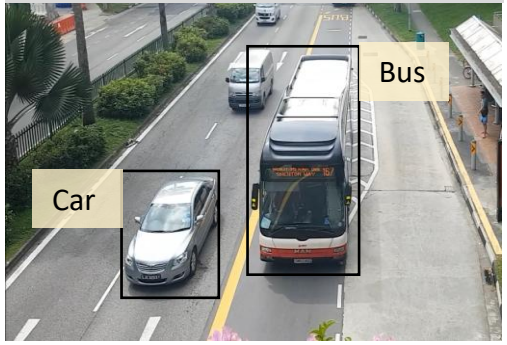
Source: https://www.bankinfosecurity.com/gao-assesses-iot-cybersecurity-other-risks-a-9926

Modern IoTs with Edge Computing

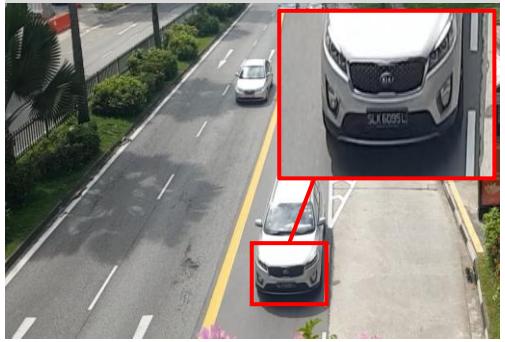
Crowd Counting



Object Localization & Classification

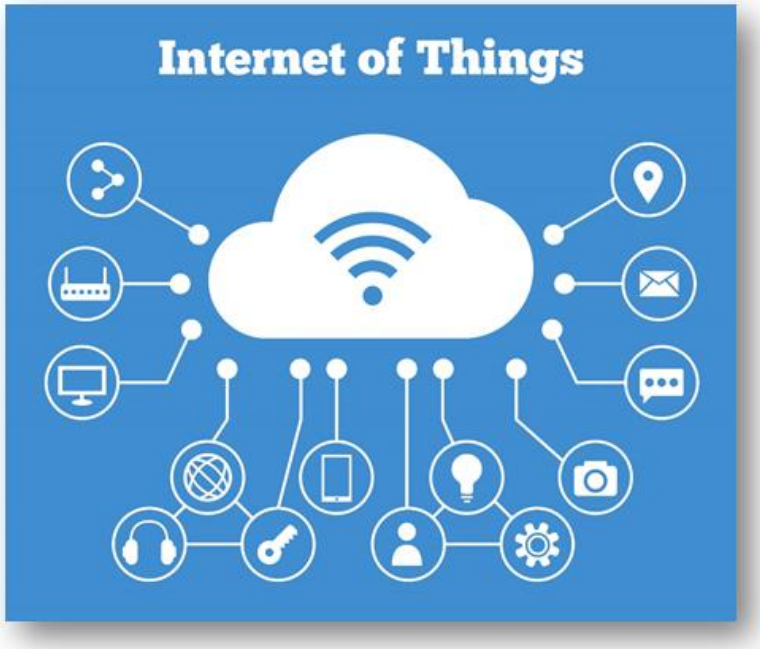


License Pate Recognition



Cloud Computing

Cloud for general computing



Cloud for IoTs

Primary difference in number of connected devices

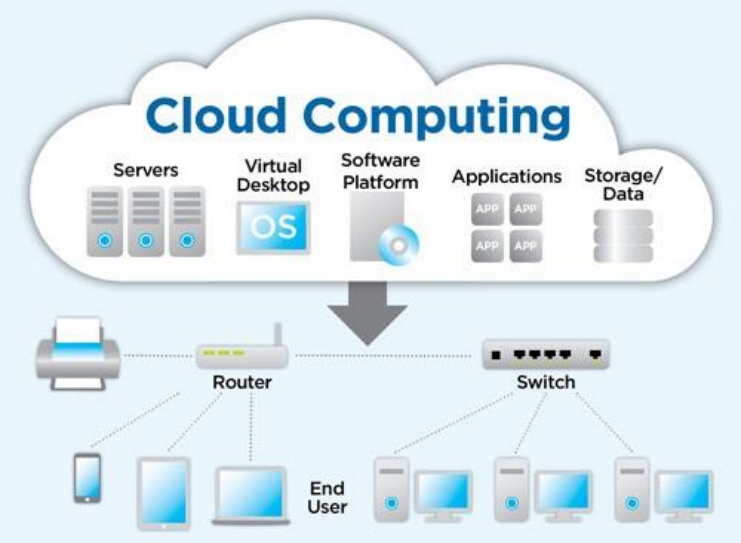
What is Cloud Computing?

Cloud computing is a model for enabling convenient, on- demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.

-National Institute of Standards and Technology (NIST)

http://www.nist.gov/itl/cloud/

Simpler definition: network-based computing taking place over the Internet, while hiding complexity of underlying infrastructure using simple APIs



- Key Characteristics:
  - On-Demand self-service
  - Scalable
  - Shared
  - Ubiquitous access
  - Rapid provisioning/released
  - Minimal management

Cloud for IoT

AWS IoT

ThingSpeak

Samsung ARTIK™ Cloud

Microsoft Azure IoT Platform

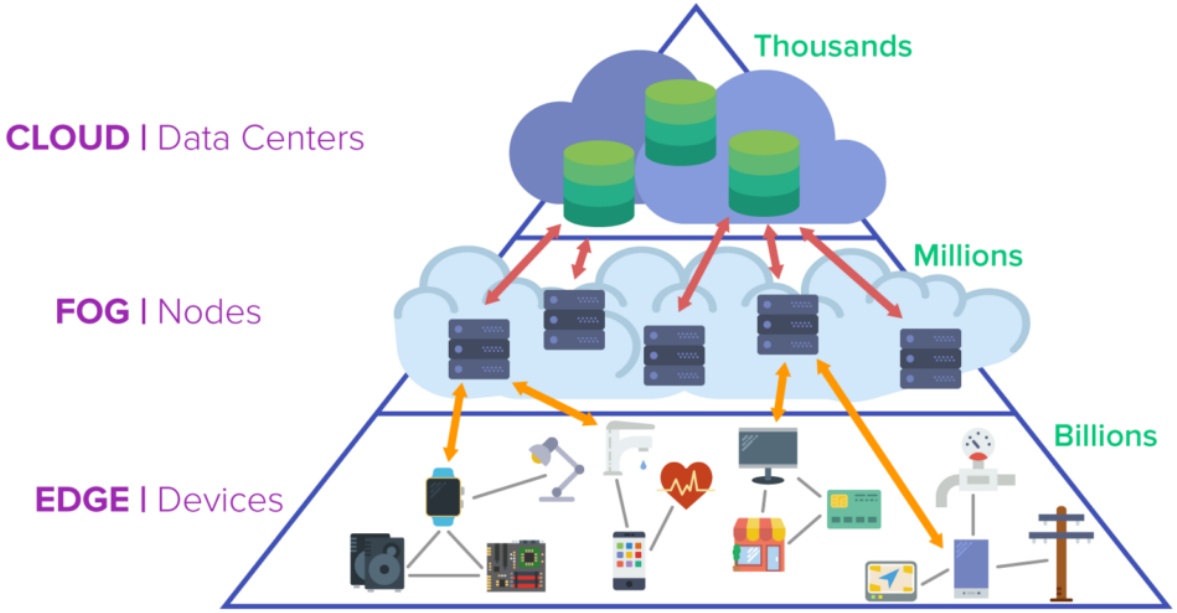
IBM Watson IoT™

Barriers to Cloud Adoption

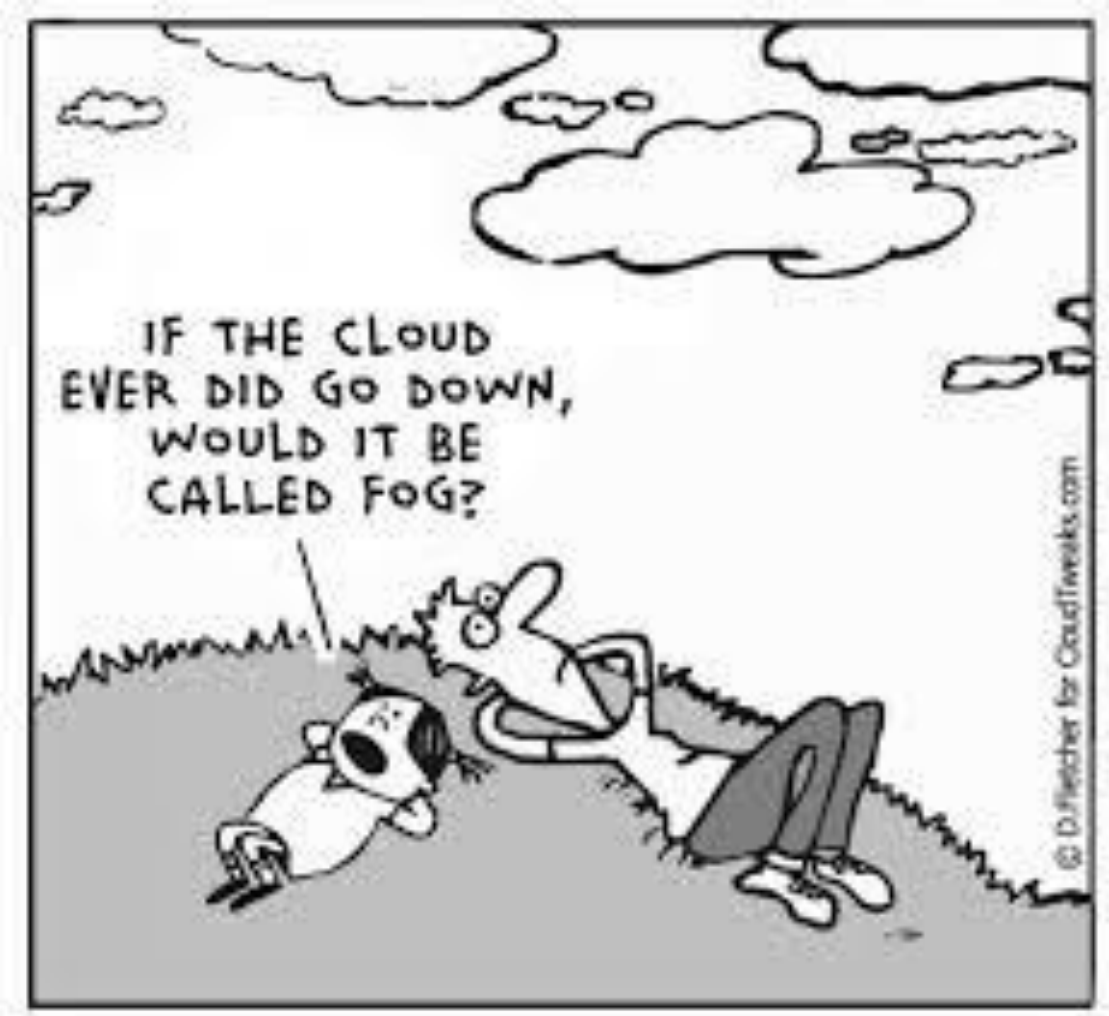
- Security:
  - Is our data secure?
  - How can we audit security?
  - Will my data be erased on deletion?
  - …
- Compliance:
  - Will we be compliant with risk, security, privacy laws?
- Interoperability:
  - Can I take my workload from provider to provider?
  - Not easily
- Service Level Management: Is the billing accurate? What happens on failures? Is the capacity sufficient?
- Tools:
  - How can I automate provisioning, monitoring, management

Ref: https://en.wikipedia.org/wiki/Cloud_computing_issues,
https://en.wikipedia.org/wiki/Cloud_computing_security

Fog Computing



Ref: https://www.omnisci.com/technical-glossary/fog-computing



Ref: http://community.spiceworks.com/topic/254392-fog-computing-replaces-cloud-as-new-tech-buzzword

Fog Computing (Cont)

- Location Aware and Location Sensitive
  - → Low latency → Computing in micro clouds
  - → Computing in the edge → Computing everywhere
  - → Fog
- Geographically distributed → Everywhere/Anywhere
- Large Scale
- Mobility
- Real-Time

Ref: F. Bonomi, et al., "Fog Computing and Its Role in the Internet of Things," ACM MCC'12, August 17, 2012, Helsinki, Finland

Summary

- **AIoT = AI + IoT** → devices become intelligent, not just connected
- **Edge Computing is essential** → reduces latency, bandwidth load, and cloud dependence
- **Massive data growth** → billions of IoT devices require local processing
- **Edge-to-Cloud networking** → mix of short-range (ZigBee/Wi-Fi) and long-range (LTE/5G)
- **Edge hardware spectrum** → SBCs (RPi), GPUs (Jetson), NPUs (Ryzen AI, Intel/Apple)
- **AI software stack** → lightweight DNNs, ONNX, PyTorch/TensorFlow, inference engines
- **Hardware acceleration** → CPU (OpenMP/NEON), GPU (CUDA/ROCm), NPU/TPU toolchains
- **Edge deployment tools** → TensorRT, TFLite, OpenVINO, DeepStream, ROCm/MIGraphX
- **Optimization** → pruning, quantization, scheduling, graph & system-level improvements
- **Real-world deployment challenges** → thermal constraints, waterproofing, power limits